

JS Modules

1. What are Modules?

- a. **Modules** are a way to split and organize JavaScript code into smaller, reusable pieces.
- b. Instead of writing all the code in one large file, you can break it into smaller files (modules) and then import/export functionalities between them.
- c. *This modular approach has several benefits:*
 - i. **Code Reusability**: You can reuse the same module in multiple parts of your application.
 - ii. **Maintainability**: Smaller files are easier to read, debug, and maintain.
 - iii. **Scoping**: Each module has its own scope, which avoids naming conflicts between variables and functions.

2. Use of **type="module"**:

- a. When working with ES6 modules in the browser, you need to specify **type="module"** in your **<script>** tag.
- b. For Example →



```
<script src="./es6Modules.js" type="module"></script>
```

- c. **Why?**
 - i. By default, browsers **do not treat** JavaScript files as ES6 modules.
 - 1. The type="module" tells the browser:
 - 2. Treat this file as an ES6 module.

3. Allow the usage of import and export keywords.
Automatically run the script in strict mode (no var hoisting, silent errors, etc.).

3. Types of JavaScript Modules:

- a. JavaScript has two primary module systems:
 - i. **CommonJS (CJS)**: Used in server-side JavaScript (e.g., Node.js). It uses `require()` and `module.exports`.
 - ii. **ES6 Modules**: Used for both front-end and back-end. It uses `import` and `export`.

4. How to Work with ES6 Modules:

- a. There are two ways to export and import in ES6 modules:
 - i. **Default Export Import**
 - ii. **Named Export and Import**

5. Default Export and Import:

- a. A module can have *only one default export*.
- b. You don't need to use *{ curly braces }* when importing it.

c. For Example →

d. index.html

```
<script src="./main.js" type="module"></script>
```

e. module1.js

```
function sum(a, b) {  
    return a + b;  
}
```

```
export default sum;
```

f. main.js

```
import sum from "./module1.js";

console.log("Main File");
console.log(sum(10, 20)); // 30
```

6. Named Export and Import:

- a. You can export multiple items from a module using named exports.
- b. When importing, you *must use the exact name* inside curly braces {}.
- c. For Example →
- d. index.html

```
<script src="./main.js" type="module"></script>
```

e. module1.js

```
// Named Exports
export let username = "John";
export const password = "123456";

function sum(a, b) {
    return a + b;
}

// Default Export
export default sum;
```

f. main.js

```
import sum from "./module1.js";
import { username, password } from
"./module1.js";

console.log("Main File");
// Default Imports
console.log(sum(10, 20)); // 30

// Named Imports
console.log("Username:", username);
console.log("Password:", password);
```

7. Importing Everything:

- You can import all the exports of a module as a single object using the *** as** syntax.
- Syntax: *** as aliasName**
- For Example →

```
import * as myModule from "./module1.js";

console.log("Main File");
// Default Imports
console.log(myModule.default(10, 20)); // 30

// Named Imports
console.log("Username:", myModule.username);
console.log("Password:", myModule.password);
```

8. **Important Note:** When you use **import * as myModule**, the default export is accessed as **myModule.default**.

9. Key Points to Remember Related to Modules:

- a. Use `type="module"` in the `<script>` tag for ES6 modules in browsers.
- b. Each module has its own scope.
- c. You can have only one default export per module but multiple named exports.
- d. *Import named exports* with `{}` and *default exports* without `{}`.
- e. Use aliases to handle name conflicts across modules.
- f. Use *as alias* to import everything from a module.